Collection Framework – Letter Combinations of a Phone Number

Observation Task: Week-13 (Experiment No: 13, CO4) Date: __ / __ / 2026

■ Objective / Aim

To write a Java program that, given a string of digits from 2-9, returns all possible letter combinations the number could represent on a telephone keypad, using the Collection Framework interfaces: List, Set and Queue (with a Map for the digit-to-letters mapping). Example: input "23" gives ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"].

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

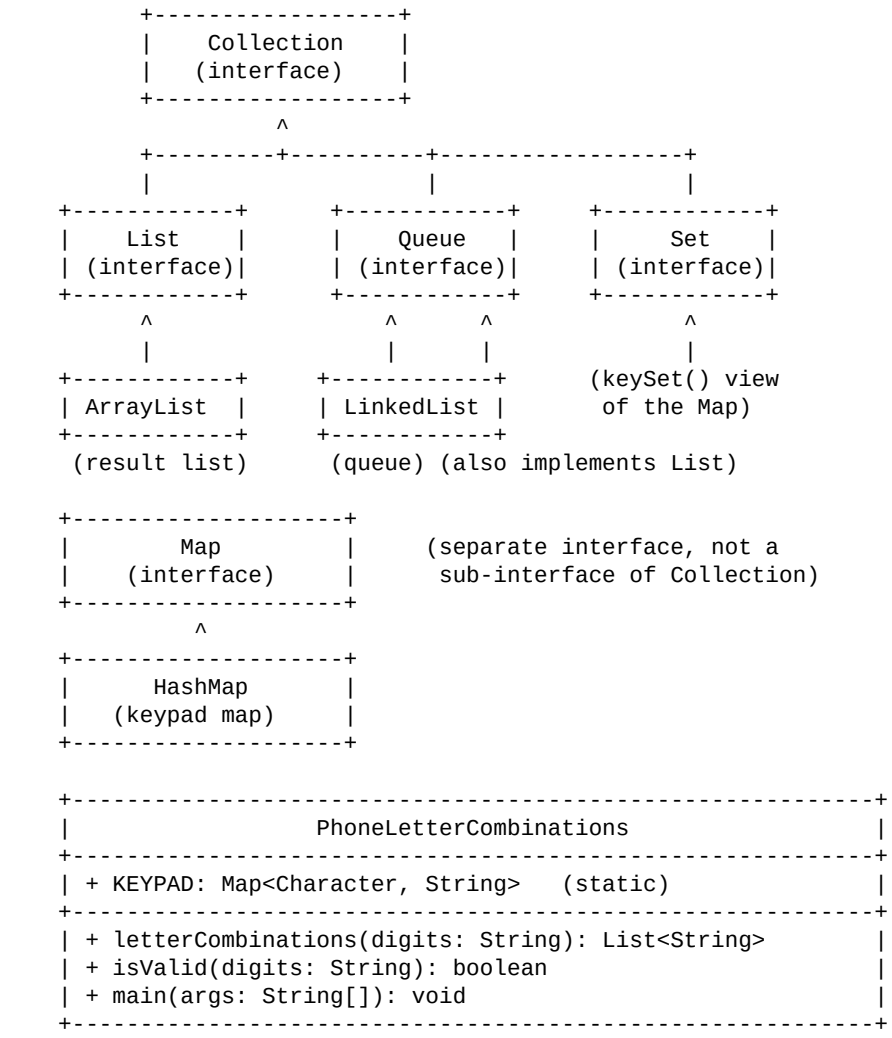
Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a class named **PhoneLetterCombinations**.
2. Store the keypad mapping in a **Map<Character, String>** (HashMap): 2 = abc, 3 = def, 4 = ghi, 5 = jkl, 6 = mno, 7 = pqrs, 8 = tuv, 9 = wxyz.
3. Define **isValid(String digits)**: use the **Set** of keys of the map (keySet()) and return false if any character is not in the set (i.e. not a digit from 2 to 9).
4. Define **letterCombinations(String digits)** that returns a **List<String>** (ArrayList):
   - If the input is empty, return an empty list.
   - Create a **Queue<String>** (LinkedList) and add an empty string "" to it.
   - For each digit, note the current queue size. Repeat that many times: remove a prefix from the front of the queue and add prefix + each letter of the digit to the back of the queue.
   - After all digits are processed, the queue holds all the combinations; copy them into the result list and return it.
5. In main(), read the digit string using Scanner, check it with isValid(), and print an error message if it is invalid.
6. Otherwise call letterCombinations(), print the list in the form ["ad", "ae", ...] and print the total number of combinations.

■ UML Class Diagram



Relationship: ArrayList implements List, and LinkedList implements both List and Queue. List, Set and Queue are sub-interfaces of Collection, while Map is a separate interface of the framework. PhoneLetterCombinations uses (association) a HashMap for the keypad, an ArrayList for the result, a LinkedList as the queue, and the Set returned by keySet() to validate the input.

■ Source Code

```
import java.util.*;

public class PhoneLetterCombinations {

    // Map (Collections Framework): digit -> letters on the telephone keypad
    static final Map<Character, String> KEYPAD = new HashMap<>();

    static {
        KEYPAD.put('2', "abc");
        KEYPAD.put('3', "def");
        KEYPAD.put('4', "ghi");
        KEYPAD.put('5', "jkl");
        KEYPAD.put('6', "mno");
        KEYPAD.put('7', "pqrs");
        KEYPAD.put('8', "tuv");
        KEYPAD.put('9', "wxyz");
    }

    // Returns all letter combinations of the given digit string
    static List<String> letterCombinations(String digits) {
        List<String> result = new ArrayList<>();      // List interface
```

```
        if (digits.isEmpty()) {
            return result;
        }

        Queue<String> queue = new LinkedList<>();     // Queue interface
        queue.add("");                                // start with empty prefix

        for (char d : digits.toCharArray()) {
            String letters = KEYPAD.get(d);
            int size = queue.size();                  // combinations built so far
            for (int i = 0; i < size; i++) {
                String prefix = queue.remove();       // take from the front
                for (char ch : letters.toCharArray()) {
                    queue.add(prefix + ch);           // add to the back
                }
            }
        }

        result.addAll(queue);
        return result;
    }

    // Checks that every character is a digit from 2 to 9
    static boolean isValid(String digits) {
        Set<Character> allowed = KEYPAD.keySet();     // Set interface
        for (char d : digits.toCharArray()) {
            if (!allowed.contains(d)) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter digits (2-9): ");
        String digits = sc.nextLine().trim();

        if (!isValid(digits)) {
            System.out.println("Invalid input. Use digits from 2 to 9 only.");
        } else {
            List<String> combos = letterCombinations(digits);
            StringBuilder sb = new StringBuilder("[");
            for (int i = 0; i < combos.size(); i++) {
                sb.append("\"").append(combos.get(i)).append("\"");
                if (i < combos.size() - 1) {
                    sb.append(", ");
                }
            }
            sb.append("]");
            System.out.println("Output: " + sb);
            System.out.println("Total combinations: " + combos.size());
        }
        sc.close();
    }
}
```

■ Compilation & Execution Commands

Save the source code exactly as:

`PhoneLetterCombinations.java`

Compile the program:

```
javac PhoneLetterCombinations.java
```

Run the compiled program:

```
java PhoneLetterCombinations
```

■ Output Screen

Run 1

```
Enter digits (2-9): 23
Output: ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]
Total combinations: 9
```

Run 2

```
Enter digits (2-9): 79
Output: ["pw", "px", "py", "pz", "qw", "qx", "qy", "qz", "rw", "rx", "ry", "rz", "sw", "sx", "sy", "sz"]
Total combinations: 16
```

Run 3

```
Enter digits (2-9): 9
Output: ["w", "x", "y", "z"]
Total combinations: 4
```

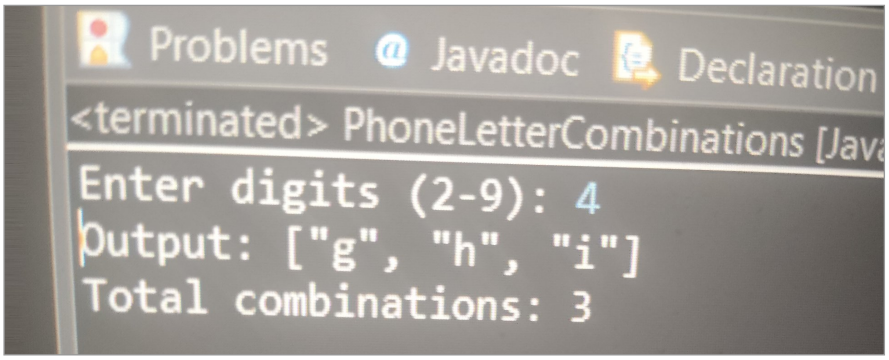
Run 4

```
Enter digits (2-9):
Output: []
Total combinations: 0
```

Run 5

```
Enter digits (2-9): 2a
Invalid input. Use digits from 2 to 9 only.
```

Run 6 (Screenshot)



■ Result

The program was successfully implemented in Java using the Collection Framework. The keypad is stored in a Map, the combinations are built using a Queue (LinkedList), the result is returned as a List (ArrayList), and a Set (the map's keySet) validates the input. For the input 23 it produced the nine combinations ad, ae, af, bd, be, bf, cd, ce, cf; an empty input gave an empty list and an invalid input gave an error message.